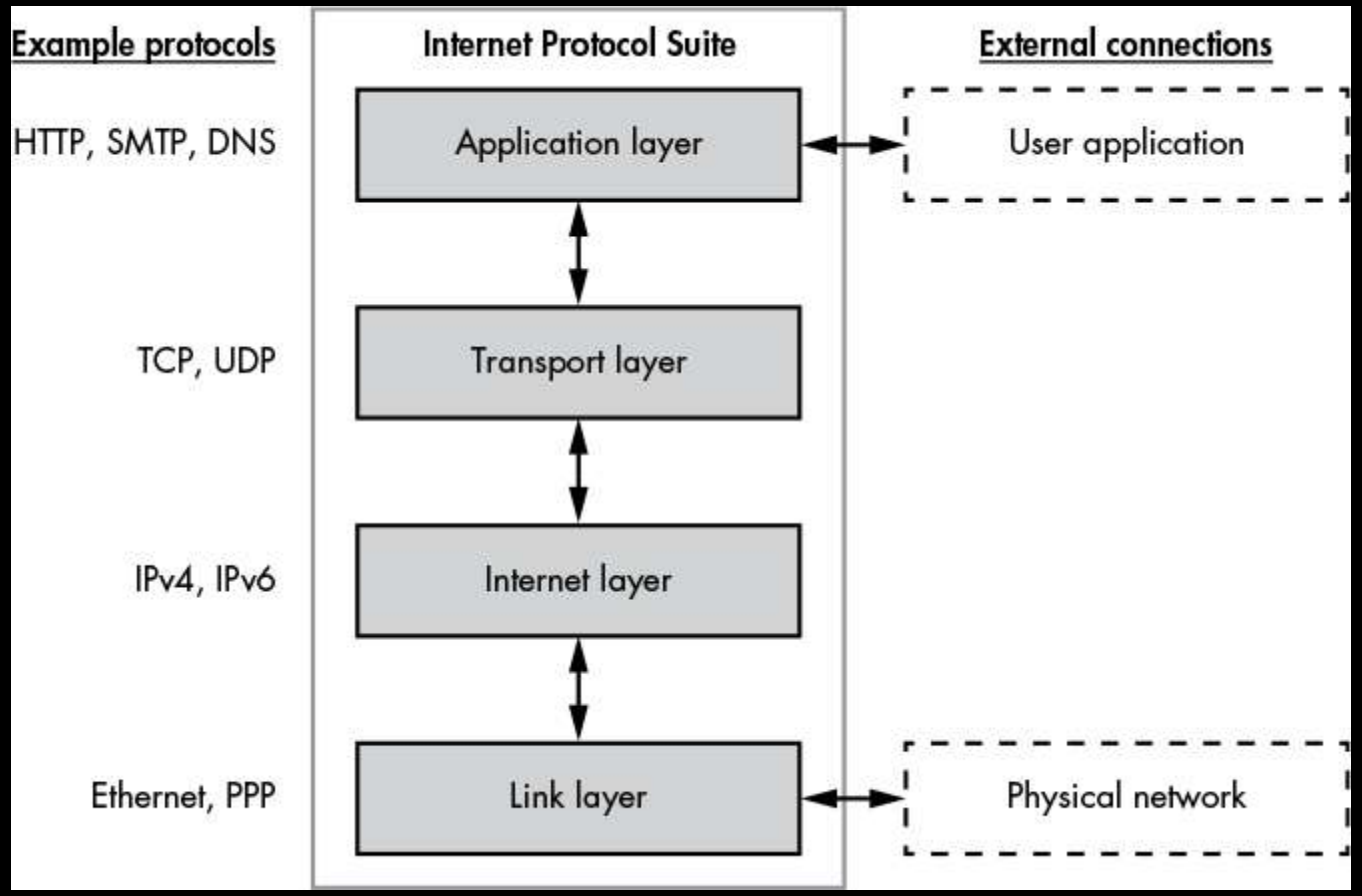
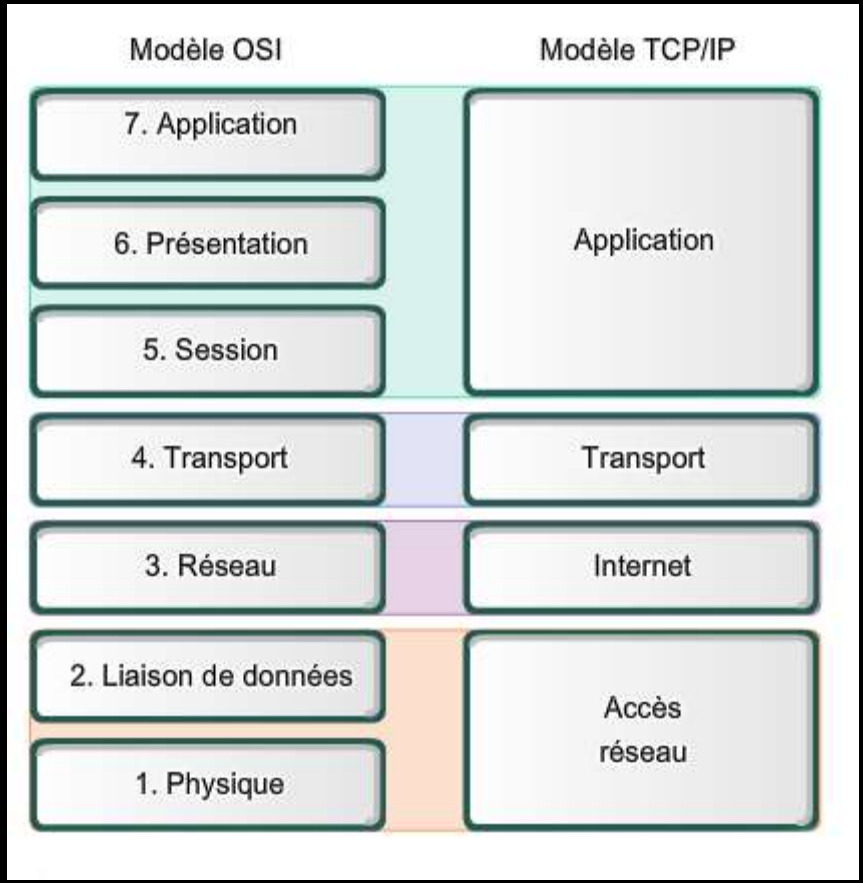
Le modèle TCP/IP

The Internet Protocol Suite

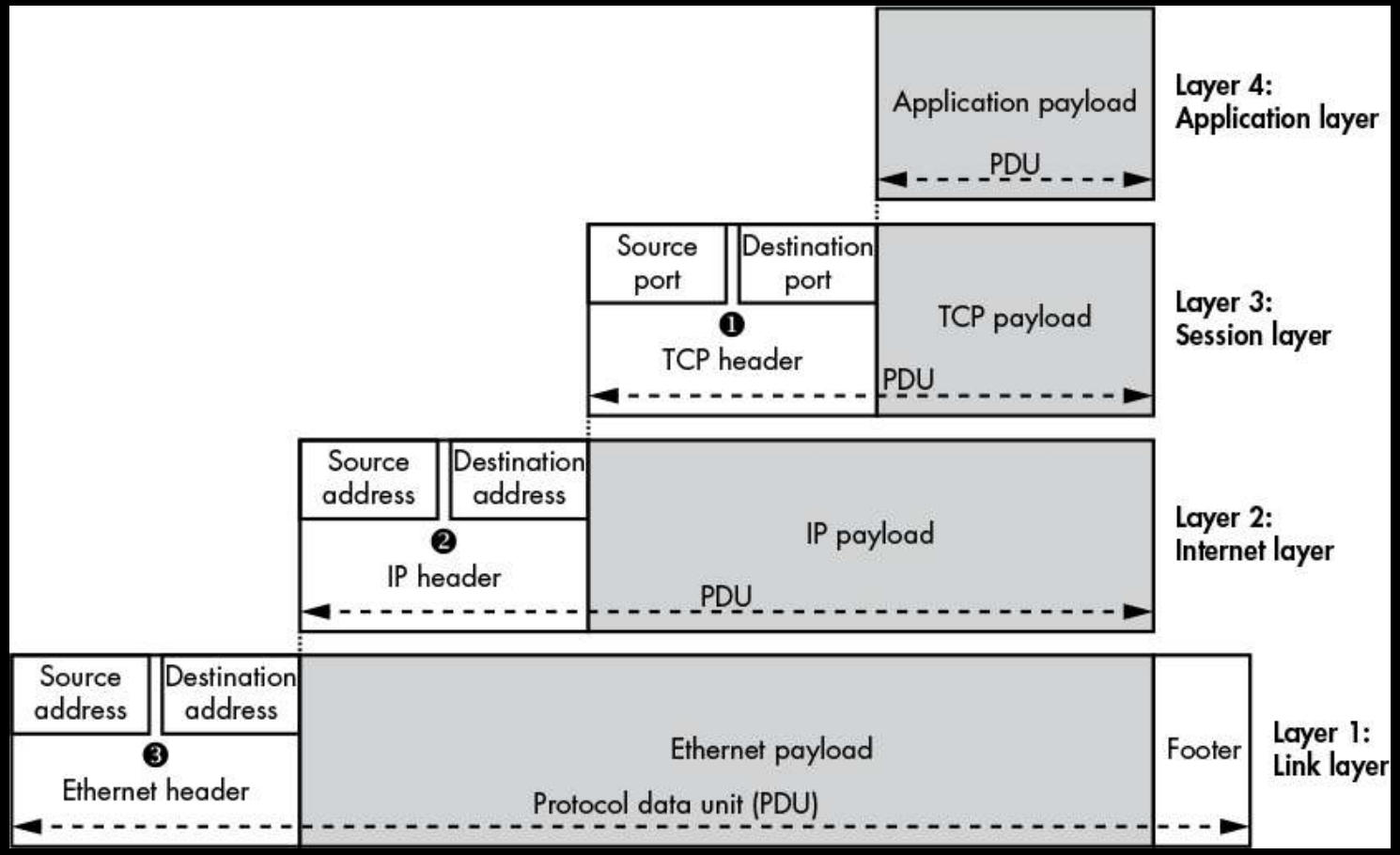
Une pile pour les diriger tous, et dans les Internet les lier



Le modèle OSI aussi utilisé



Encapsulez-moi tout ça

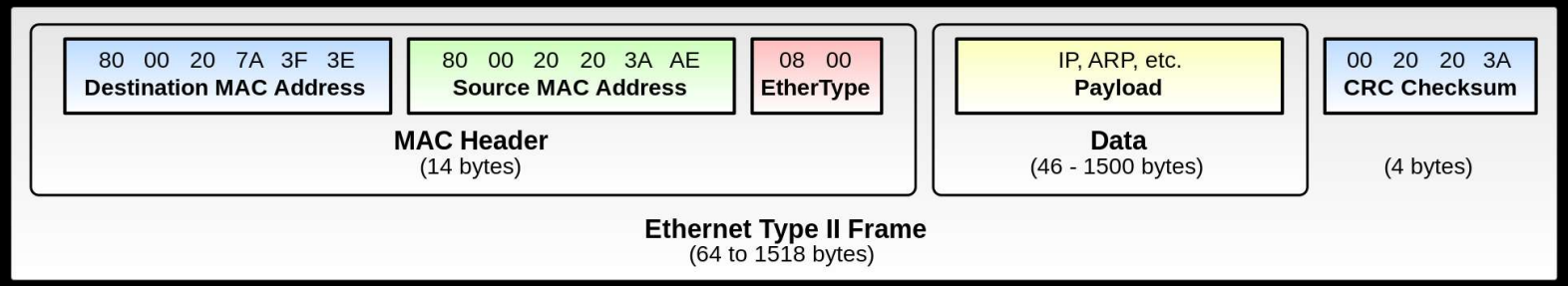


C’est troué tout ça…

- Par défaut, tout circule en clair. On peut donc tout observer à l’aide de Wireshark par exemple.
- Il y a très peu de systèmes de vérification notamment d’authenticité de la source.
- Comment exploiter tout ça?

Tu as le lien?

- Principal protocole : Ethernet
- Adresse MAC : 6 octets. Une adresse de broadcast : FF:FF:FF:FF:FF:FF
- Composition d'une trame Ethernet



- Une adresse MAC peut être modifiée très facilement :

macchanger –mac=AA:BB:CC:DD:EE:FF interface

Allô? J’ai le bon numéro?

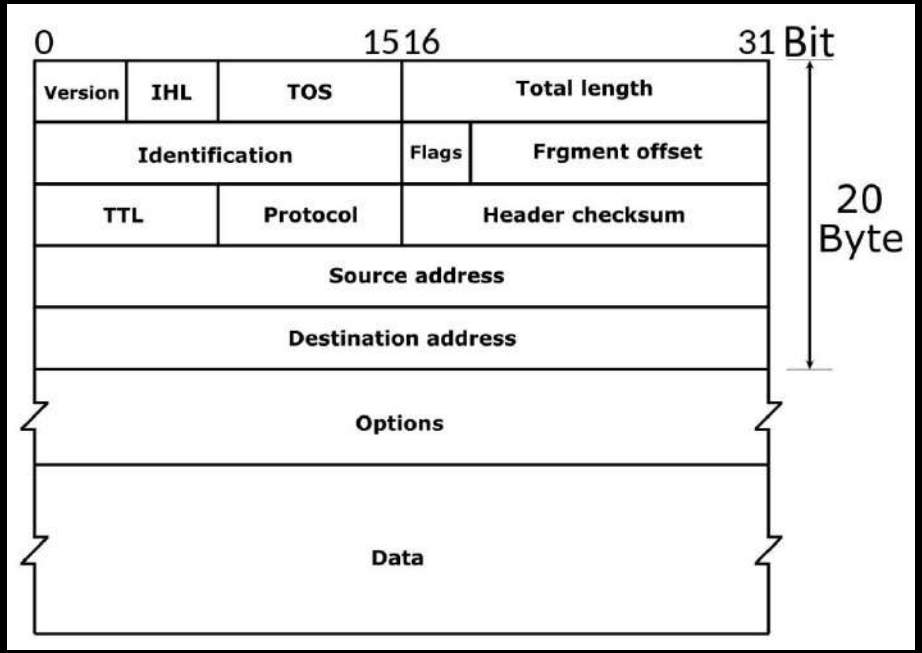
- Pour faire le lien entre IP et MAC : le protocole ARP
- 2 étapes :
  - Broadcast : Qui a telle IP?
  - Réponse : J’ai telle IP et j’ai telle MAC
- Aucun moyen de vérifier si une réponse a été demandée. On peut donc directement forger les requêtes ARP et se faire passer pour un autre ordinateur : c’est l’ARP spoofing
- On peut ainsi remodeler complètement le réseau et se placer comme centre du réseau et faire en sorte que tout passe par nous
- nemesis arp –v –r –d interface –S IP_source –D IP_destination –h MAC_expéditrice –m MAC_cible –H MAC_source –M MAC_destination
- arpspoof –i interface –t cibles hôte_cible

Quelques applications

- Pouvoir observer tout le trafic sur le réseau
- Faire office de proxy et donc pour pouvoir potentiellement décrypter le trafic chiffré :
  - Principe simple : servir sa propre clef aux deux extrémités. Ne marche pas avec HTTPS car l'authenticité des certificats sont vérifiés. Marche cependant avec ssh. La preuve par l'exemple…
    - ssh-mitm

Comment je vais sur Internet dans tout ça?

- IPv4 et IPv6
- IPv4 :
  - Quelques IPs réservées au local : 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/12
  - On ne peut vraiment s'attribuer n'importe quelle IP
- Trame IPv4 :

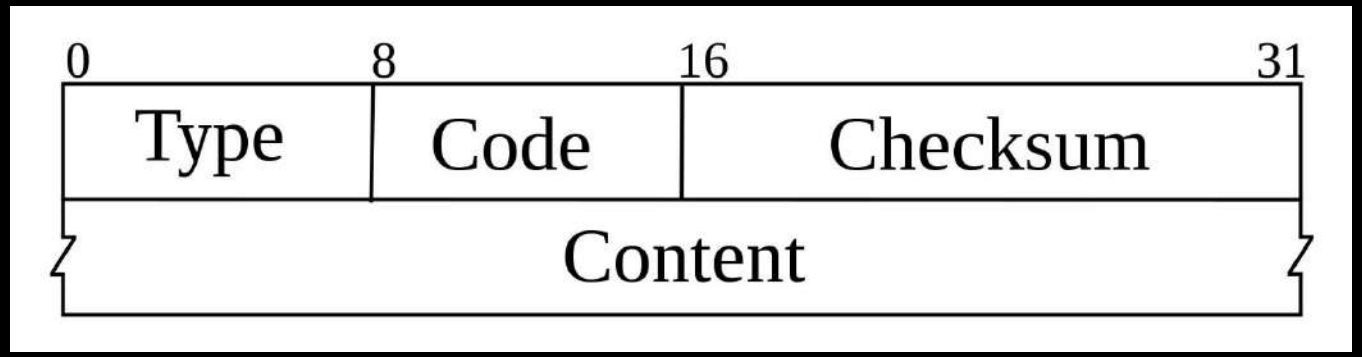


Et comment je transporte ça?

- Trois protocoles principaux :
  - TCP : garanti l'arrivée des données, qu'elles soient replacées dans le bonne ordre et gère la congestion sur le réseau
  - UDP : plus léger mais aucune garantie que les données arrivent. Permet aussi de construire son propre protocole de transport par-dessus simplement (comme pour HTTP 2 par exemple). Prioritaire sur TCP
  - ICMP : ping mais aussi message d'erreur

Ping? Pong.

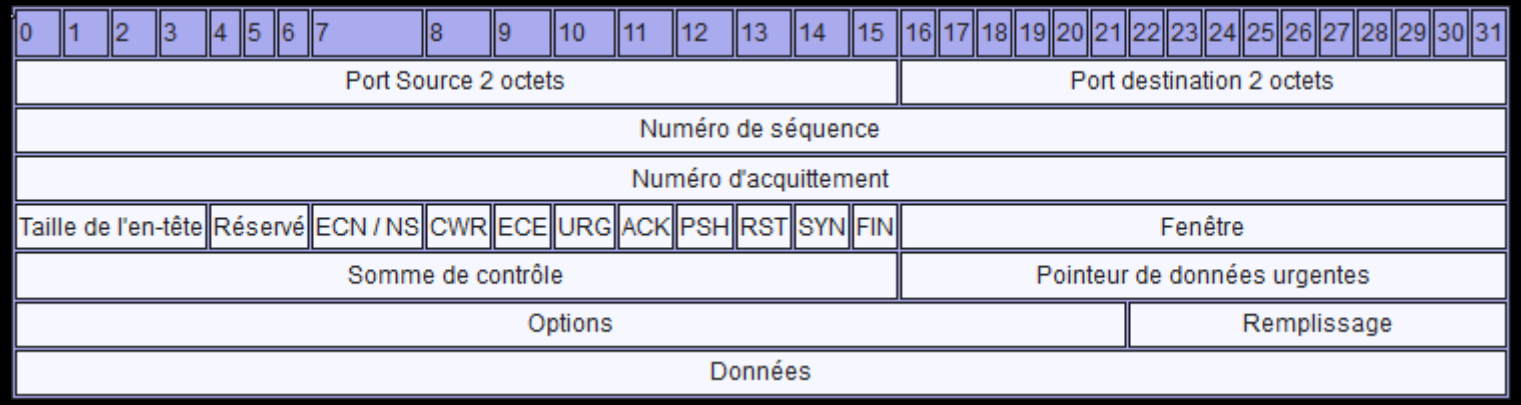
- Header ICMP :



- Ping of death :
  - Un ping ne peut transporter que 2^16 octets de données. Sur certaines vieilles implémentation, un ping qui transporte plus de données peut faire crasher la machine

Allô? Allô, j’ai bien reçu ton allô. Moi aussi, on peut maintenant parler!

- Header TCP :



- Attaque DOS par SYN flooding
- On peut aussi détourner un flux TCP en envoyant un drapeau RST puis en interceptant le 3-way handshake TCP
- Peut servir à scanner les ports d’une machine

La brique de base

- Header UDP :

Port Source (16 bits)	Port Destination (16 bits)	Longueur (16 bits)	Somme de contrôle (16 bits)	Données (longueur variable)

- Problème : comme il n'y a aucune mise en place de session, il est très simple à utiliser pour faire des DOS avec des grands facteurs d'amplification :
  - Exemple : faire des requêtes DNS mais rediriger la réponse vers la machine cible
  - DDOS de DynDNS en 2016 : des centaines de milliers de machines ont flooder les serveurs DNS de DynDNS d'un nombre très important de requêtes UDP (~1Tbps) qui ont fini par faire tomber les machines pendant plusieurs heures.

Mon application, enfin !

- Dans la couche application se situent les données qui doivent être transportées et qui nous intéressent.
- Leur format va dépendre du type de données et du protocole utilisé pour les transporter (HTTP, FTP, POP, IMAP, SMTP, etc.)

Je t’ai vu !

- La plupart des attaques citées sont visibles car génèrent beaucoup de paquets d’un même type
- Des contre-mesures existent et se basent sur ce nombre anormal de requêtes d’un même type pour détecter les attaques
- Il faut donc être suffisamment discret pour ne pas se faire repérer et se faire bloquer